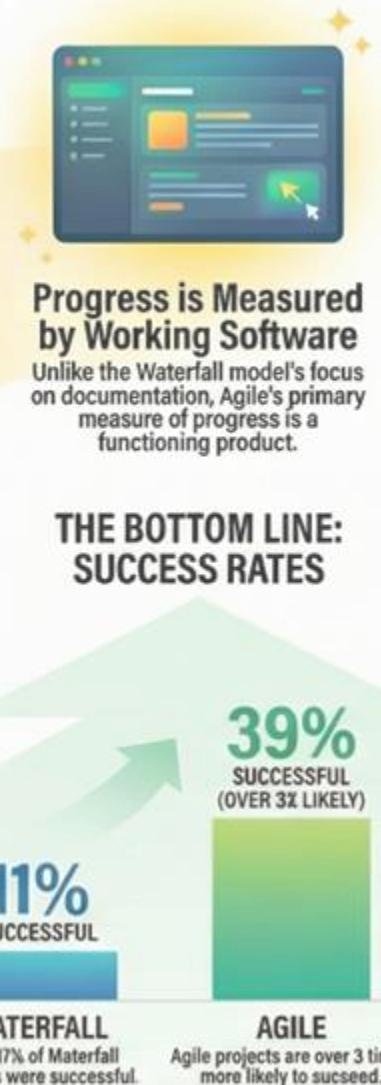
# Development

Martin Benning
(University College London)










ENGF0034 - Design and Professional Skills

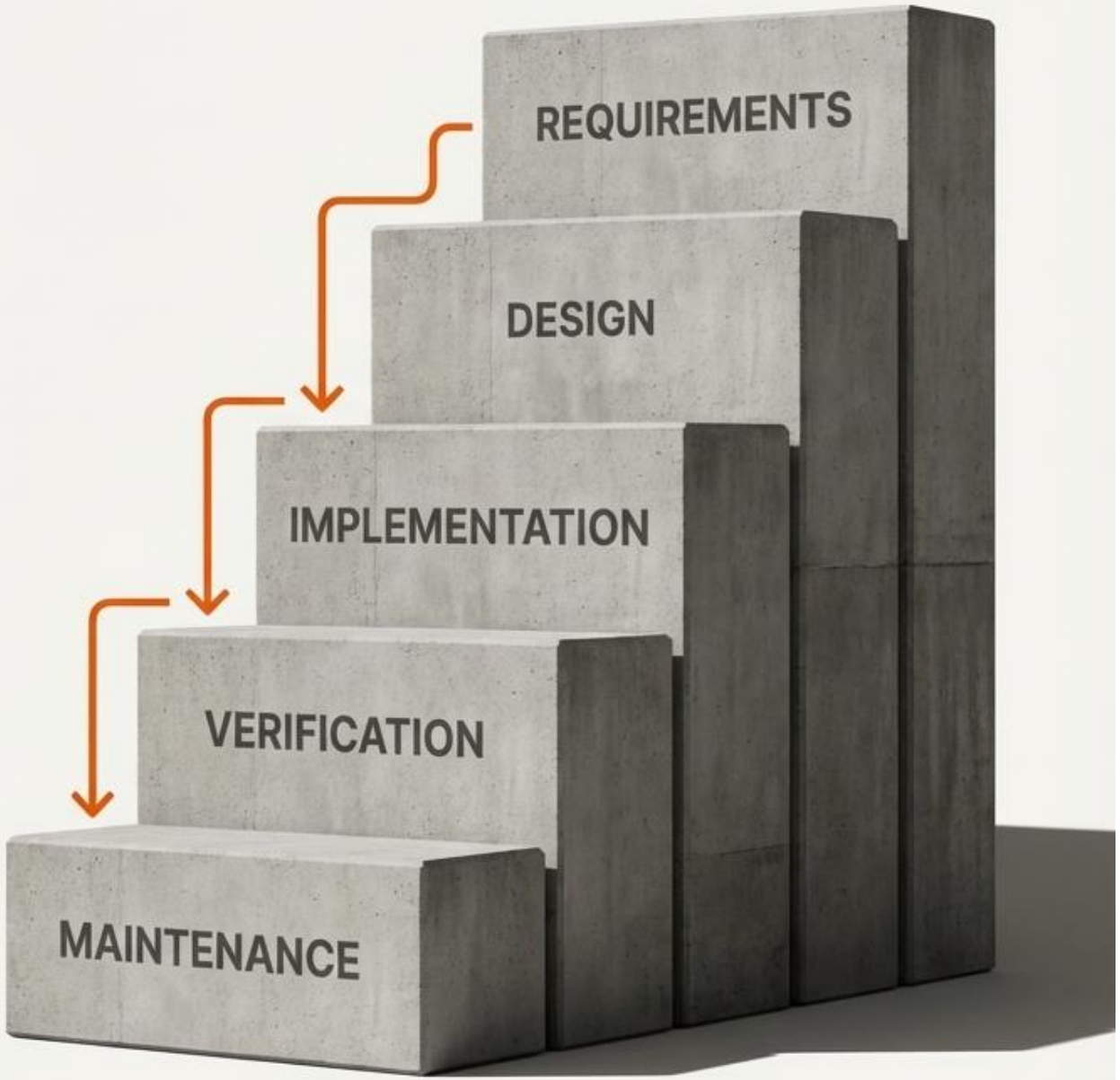
# The Challenge of Collaboration

Writing software alone is manageable. Building large systems as a team is where **most projects fail**. Our industry has a terrible record for failed software projects with large teams.

**How do we organize teams to build large, successful systems that users actually want?**

# The Waterfall Model: A Legacy from Mature Engineering

A sequential, top-down process where the output of one stage cascades directly into the next.




“Borrowed from other, more mature fields of engineering.”

Join at menti.com | use code 4411 5756

What might make the waterfall model a good or a bad choice for software engineering?

All responses to your question will be shown here

Each response can be up to 200 characters long

Turn on voting to let participants vote for their favorites

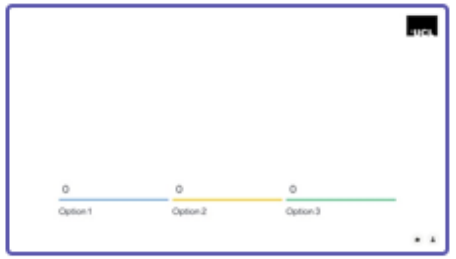
Show correct answer

MB

Menti

ENGF0034 2025 - ...

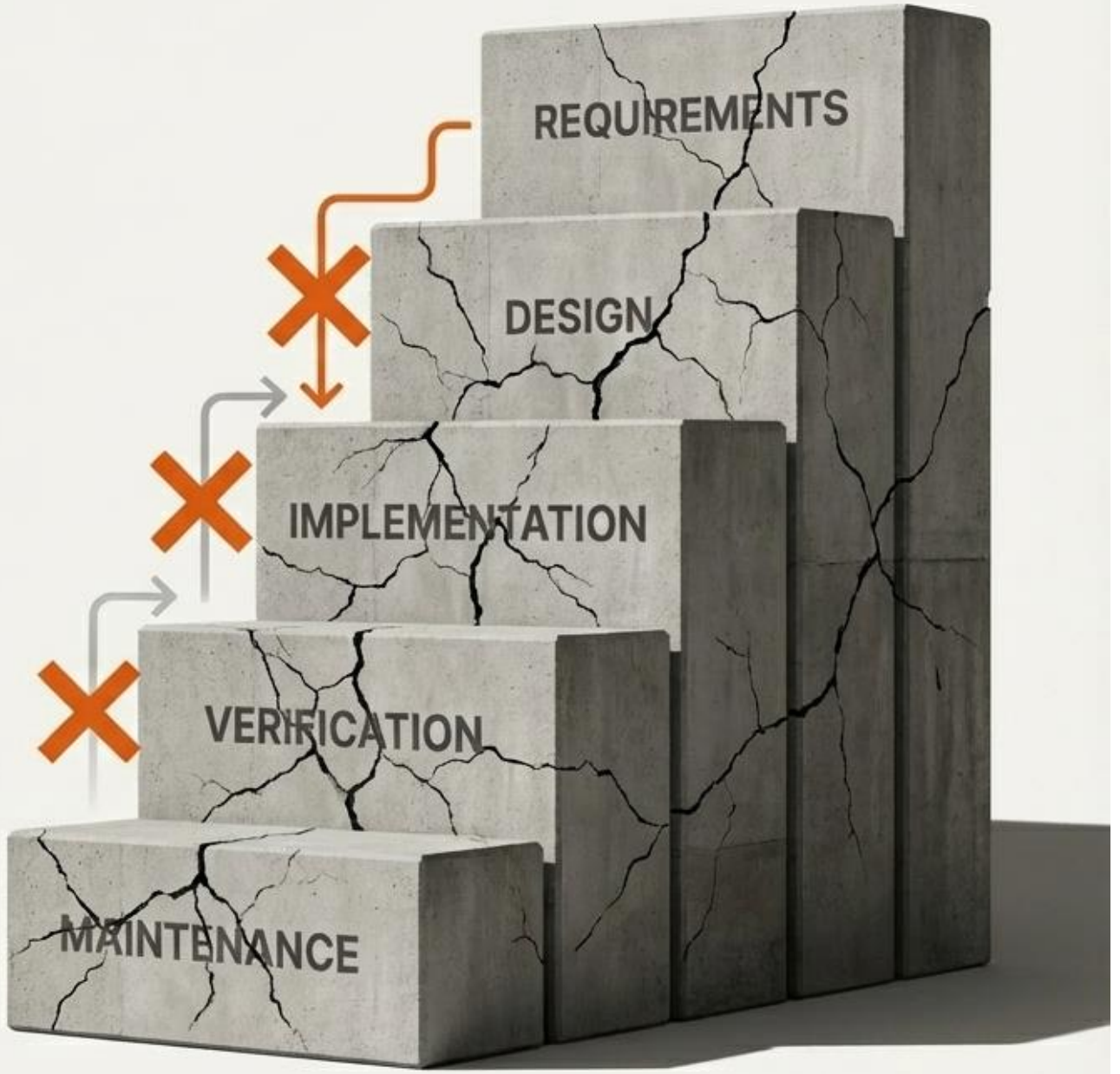
Choose a slide to present



# Why the Waterfall Fails for Software

Software's unique properties—its flexibility and complexity—shatter the linear assumptions of traditional engineering.

- **Unstable Requirements:** Users often don't know what they want until they see a prototype. Software isn't really constrained by physical limits.
- **Intrinsic Complexity:** The interaction between software components is immensely complex and difficult to specify fully upfront.
- **No Separate Manufacturing Step:** A testing-phase discovery can force a restart from the requirements phase.



# The Software Tar Pit

Analogy from Frederick Brooks' "The Mythical Man-Month."

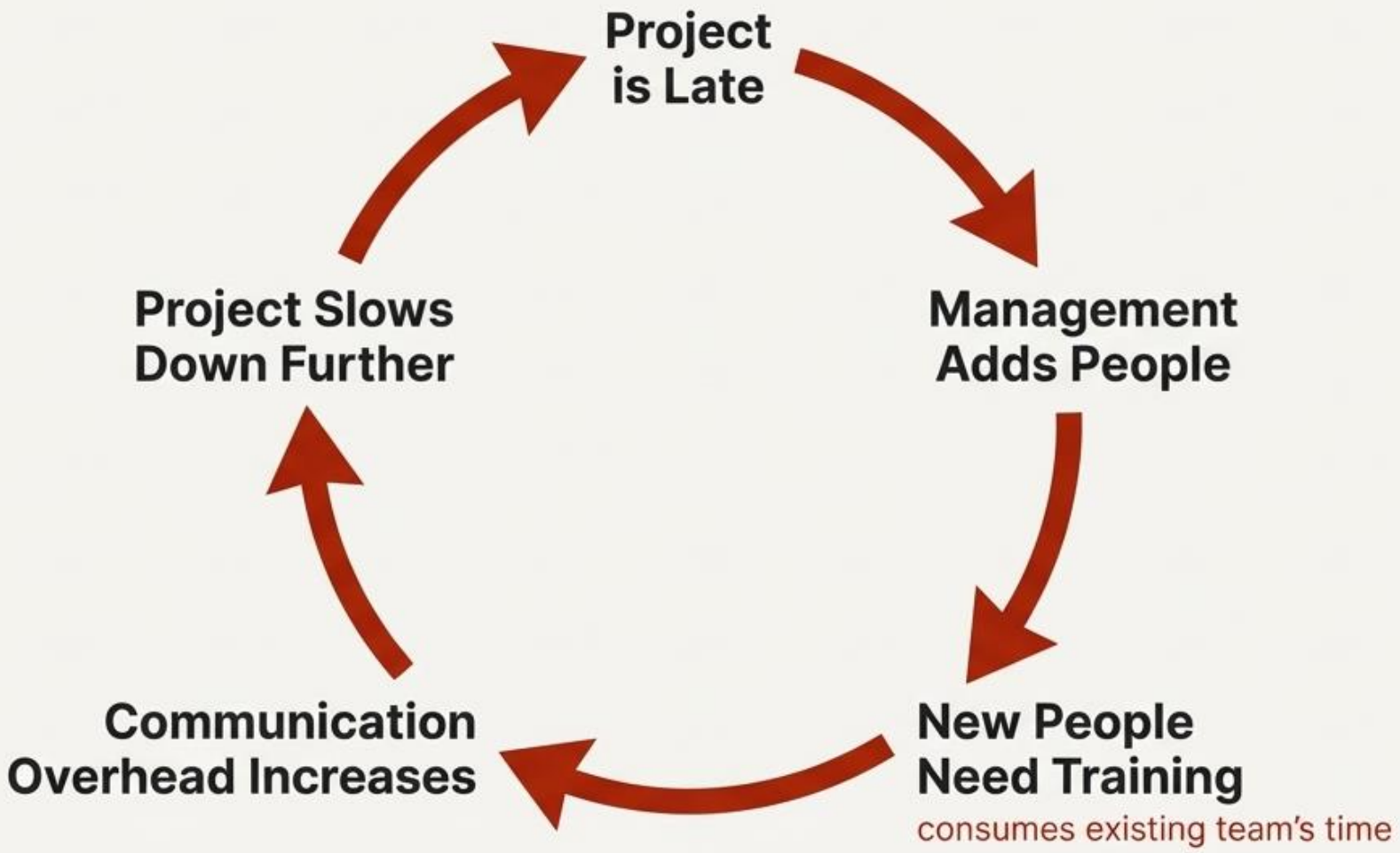
When a project falls behind, the intuitive response—adding more people—only makes it sink faster.

1. Project is observed to be late.
2. Management adds more developers.
3. New developers require training, slowing the existing team.
4. Communication overhead explodes. The project falls even further behind, sucking in more resources until it dies.

UCL


The trap preys on ignorance.
The struggling mammoth attracts other animals—predators and scavengers—looking for an easy meal. But the same dangers that trapped the mammoth are still present.

# More resources create a death spiral.



Join at menti.com | use code 4411 5756

# What could be better approaches?

All responses to your question will be shown here

Each response can be up to 200 characters long

Turn on voting to let participants vote for their favorites

→ Show correct answer

MB

**Menti**

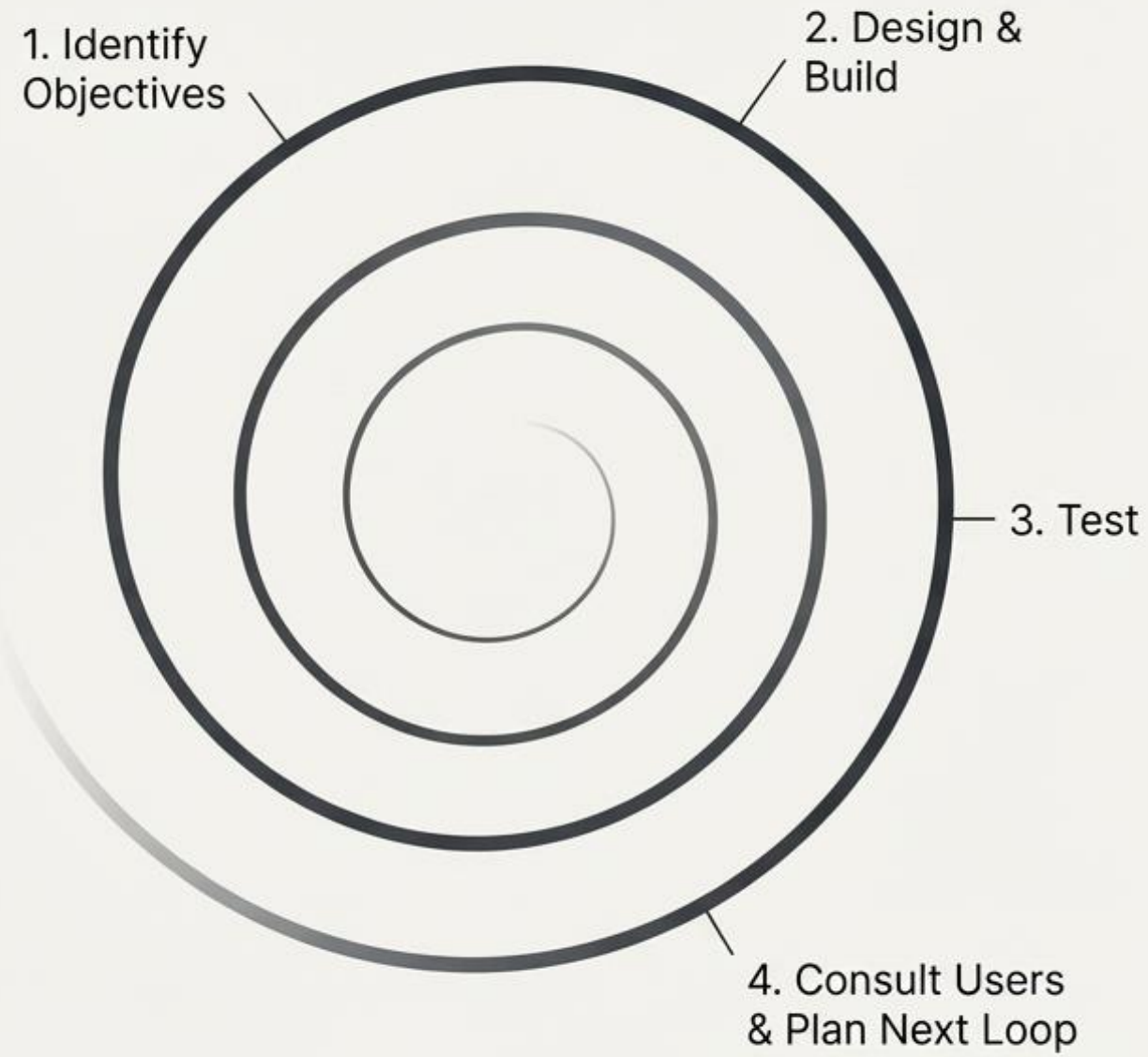
ENGF0034 2025 - ...

**Choose a slide to present**


# The Escape: Iterative Development via the Spiral Model

Instead of one giant, risky leap, we take many small, safe steps, ensuring we always have a working system.

**Go from a working system to a *better* working system in short, repeatable cycles.**



# Codifying the Escape Plan: The Agile Manifesto

## 1.

### Satisfy the Customer

Through early and continuous delivery of valuable software. Welcome changing requirements, even late in development.

## 2.

### Working Software is the Measure

Deliver working software frequently, from a couple of weeks to a couple of months. This is the primary measure of progress.

## 3.

### Trust Self-Organizing Teams

Build projects around motivated individuals. The best architectures, requirements, and designs emerge from teams you trust to do the job.

# The Four Components of an Agile Team.

### Project Leader

Maintains the complete vision. Ensures cohesion in the final product.

### On-Site Customers

Represents the customer's view in all design decisions. Ensures the team builds what is actually wanted.

### Programmers

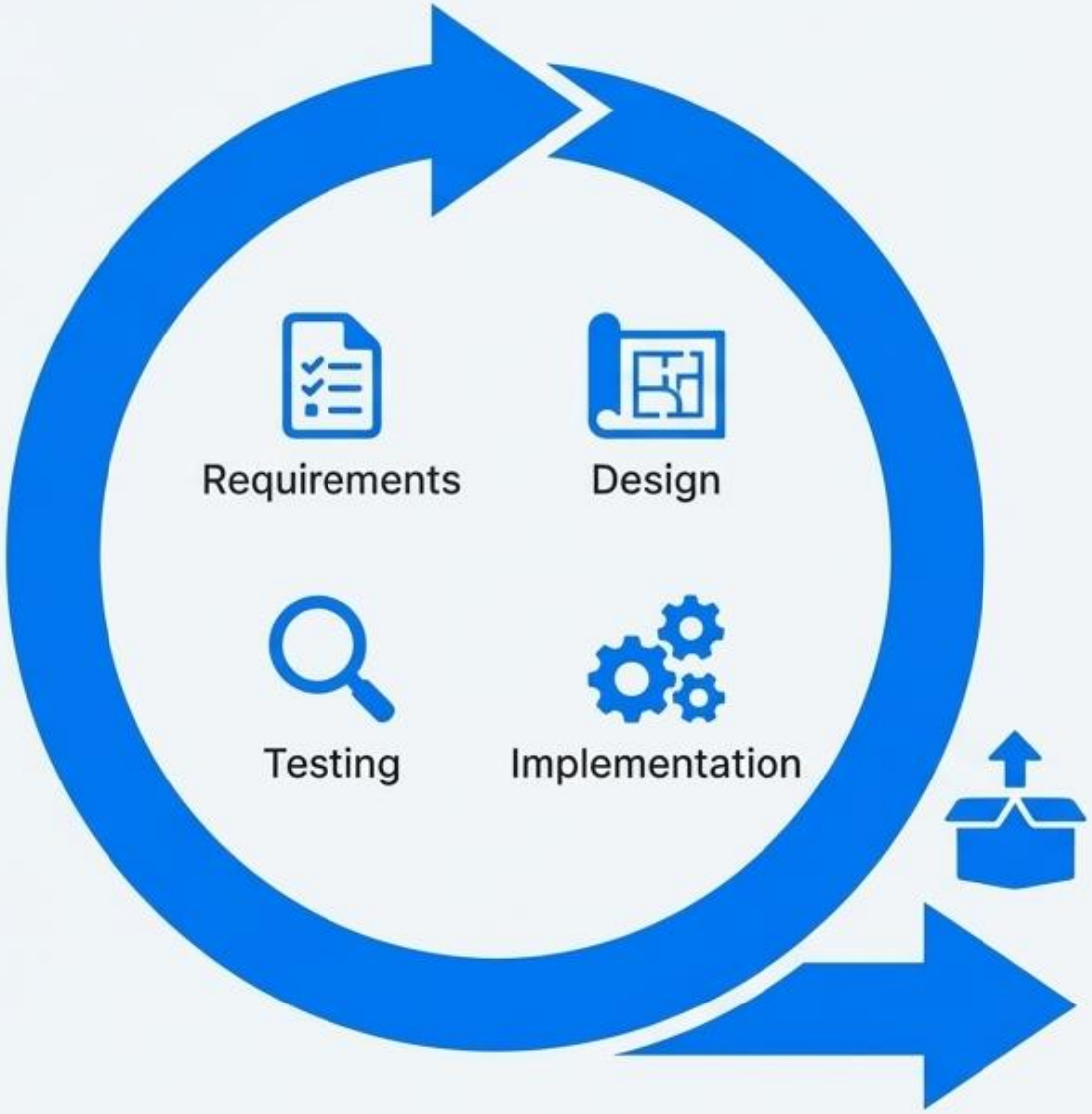
Organized into small, effective teams (typically 4-9 people) to minimize communication overhead and maximize focus.

### Domain Experts

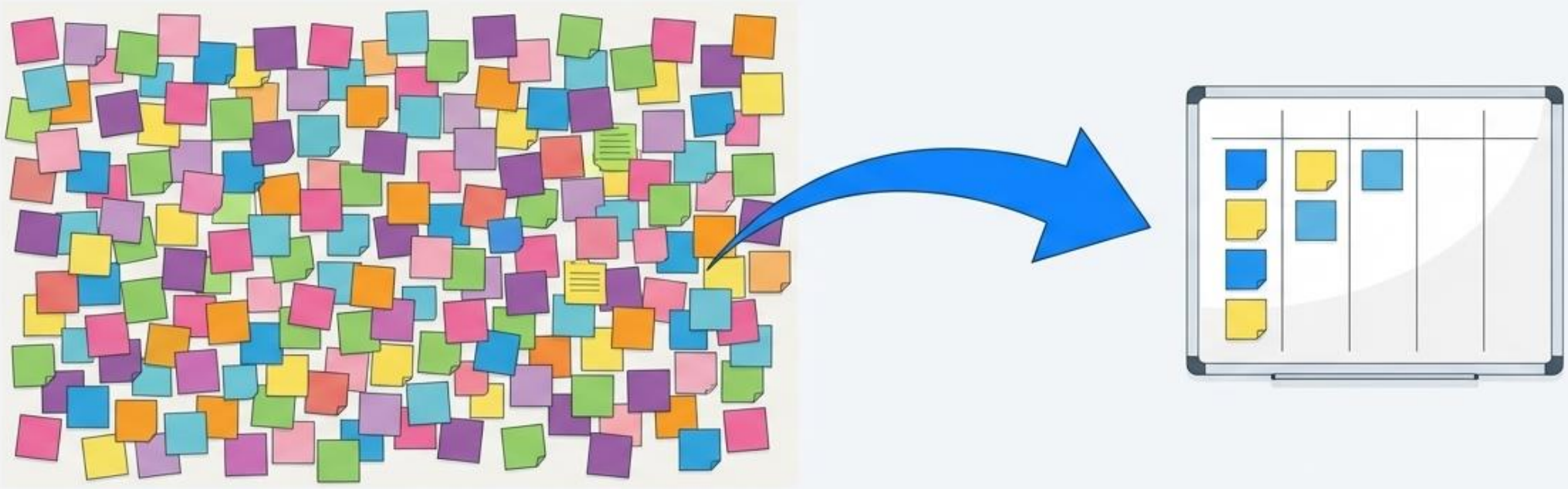
Specialists (e.g., in security or user interfaces) consulted when specific, deep expertise is needed.

# The Rhythm of Progress: Code Sprints.

- **Time-boxed:** Typically two weeks long.
- **Complete Cycle:** Within each sprint, you perform requirements, design, implementation, and testing.
- **The Goal:** Deliver software that has gained the intended feature and is potentially releasable.



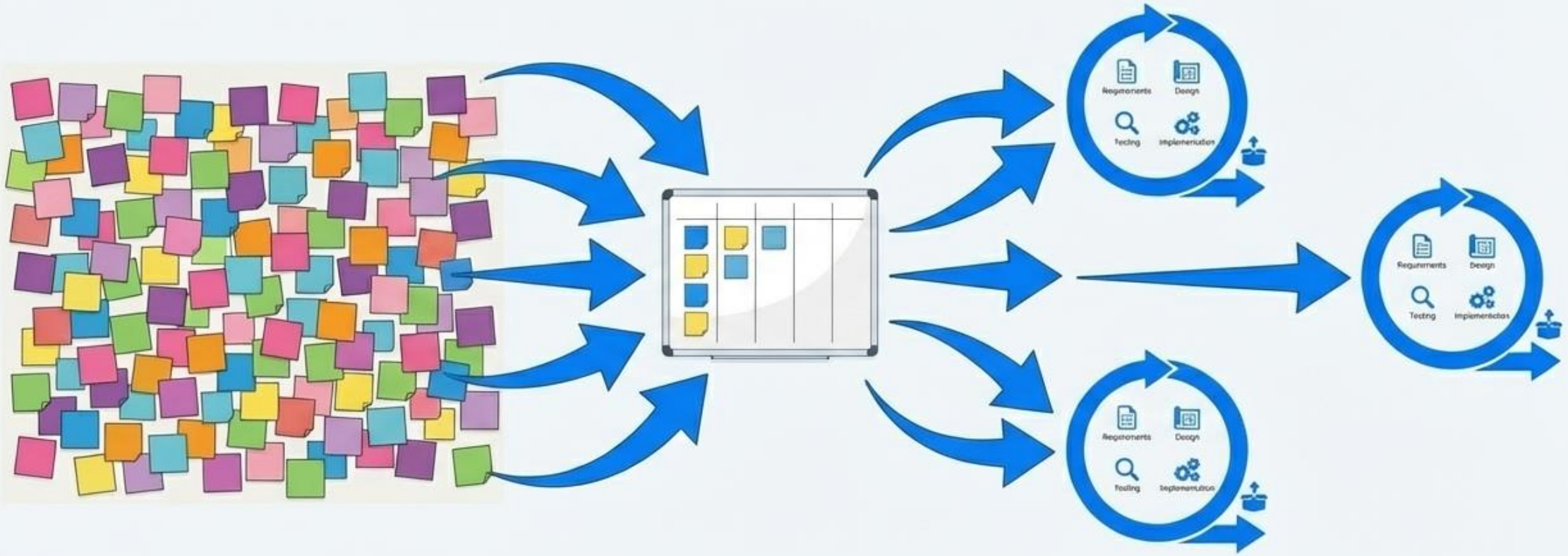
# Charting the Course: From Ideas to Action.



**The Task Board:** A collection of every possible “user story” or feature. A brainstorm of potential value, including ideas you may never build.

**The Backlog Board:** When planning a release, selected features are moved here. This board dictates the tasks to be worked on during sprints.

# The Agile Workflow in Action.



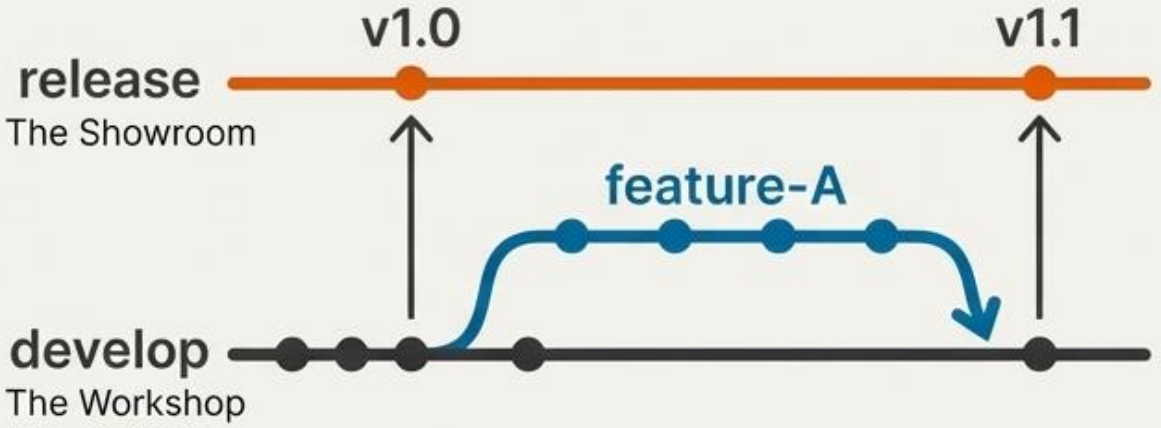

A large team can run multiple sprints from the backlog simultaneously. This creates massive parallelism in the development process, guided by a single, prioritized plan.

# The Engine of Collaboration: Git

Git is the version control system that allows parallel development without chaos through a system of branches.

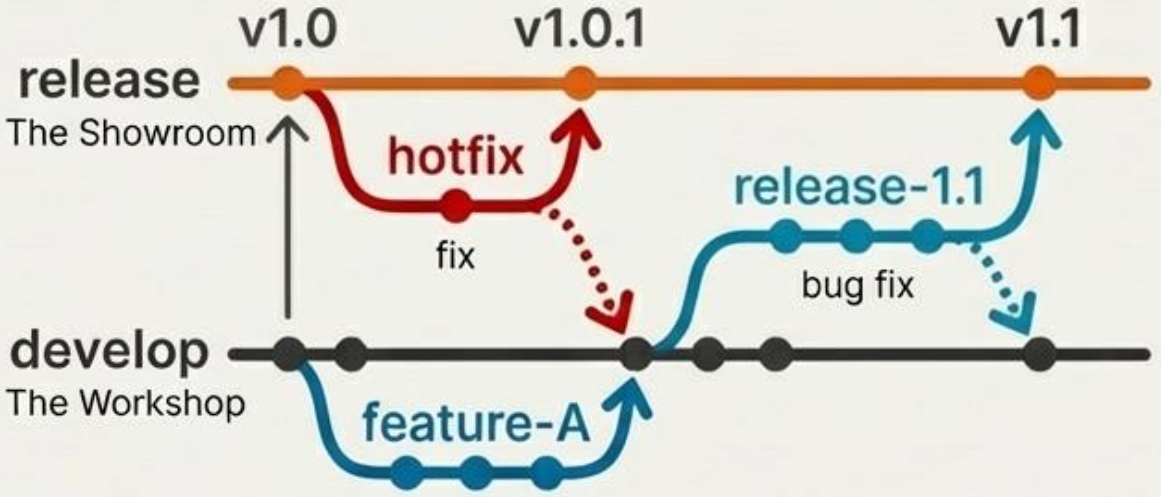
# Parallel Universes: Feature Branches

Feature branches let teams build and test new ideas in isolation, ensuring the main development branch is always stable. The main development branch always works and always passes all of the unit tests. No one is disrupted by half-finished work.



# Professional Workflow: Release & Hotfix Branches

Specialized branches allow for careful release preparation and emergency bug fixes without disrupting the development rhythm.

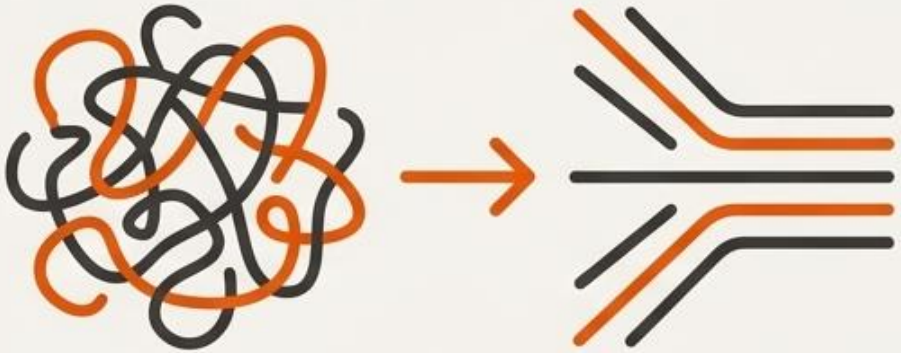


# The Safety Net: Continuous Integration (CI)

Every time code is committed to the `develop` branch, an automated system builds and tests it on all platforms to catch bugs instantly.

# Honing the Craft: Advanced Techniques

Agile is a mindset of continuous improvement, supported by practices that maintain long-term code quality and collaboration.



## Refactoring

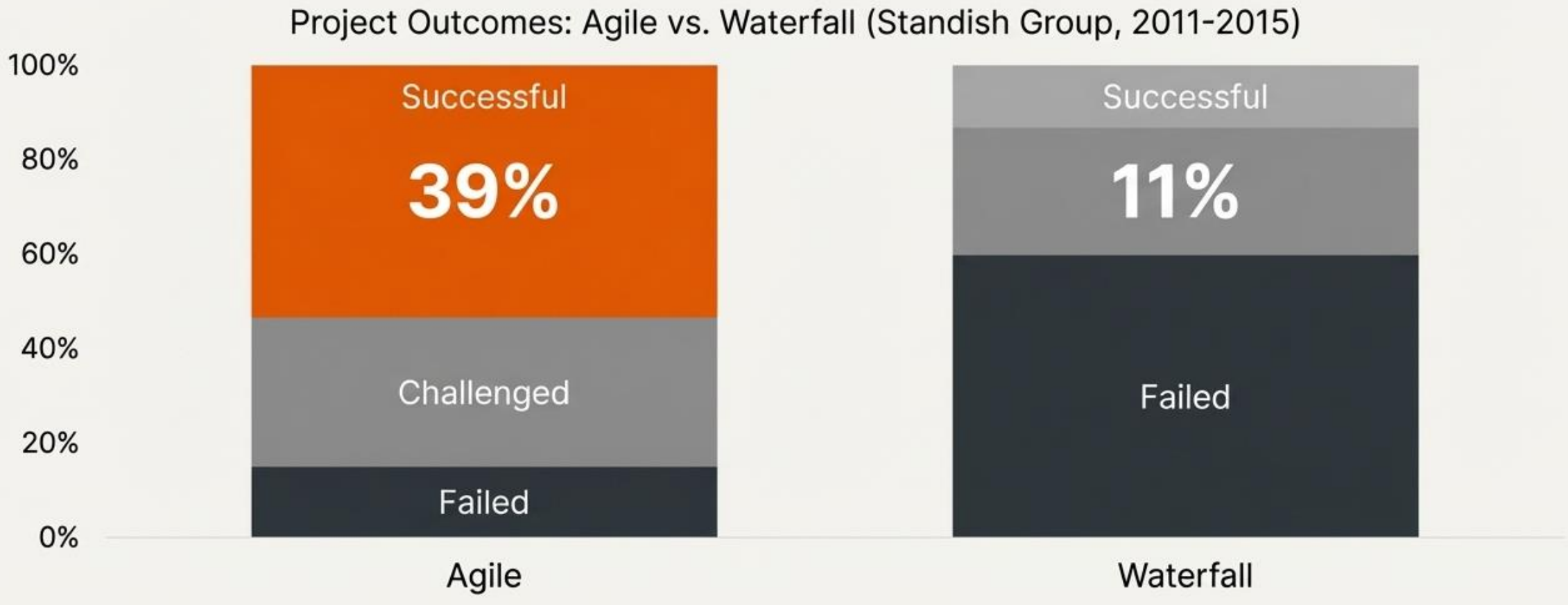
The process of restructuring existing computer code without changing its external behavior. Made safe by a strong suite of automated tests from CI.

## Pair Programming

Two programmers, one computer. One "drives" by typing, the other "navigates" by thinking. Leads to higher quality code and shared knowledge.

# The Hard Truths of Software Development

Developing software is really hard. Agile dramatically improves the odds, but it is not a magic bullet.



Join at menti.com | use code 4411 5756

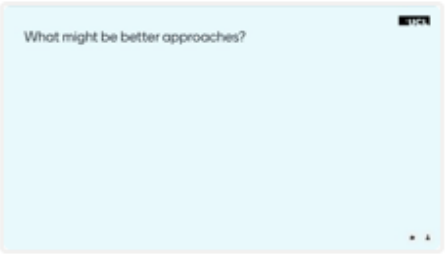
# Which of these components is **not** a component of an agile team?

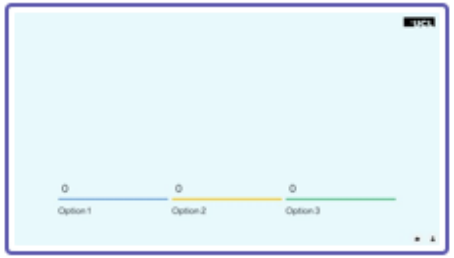
| 0 | 0 | 0 | 0 |
|---|---|---|---|
| On-site Customers | Independent Validation and Testing Team | Lead Systems Design and Specification Team | Domain Experts |

Menti

ENGF0034 2025 - ...

Choose a slide to present



Join at menti.com | use code 4411 5756

Rank the following four items in order of priority, according to the principles of the Agile Manifesto

Comprehensive documentation

Face-to-face communication

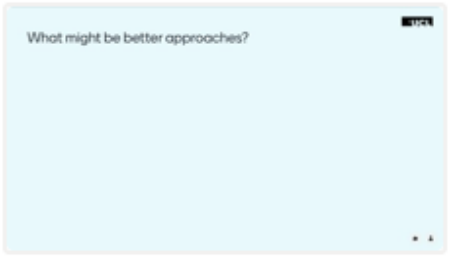
Strict adherence to the original requirements

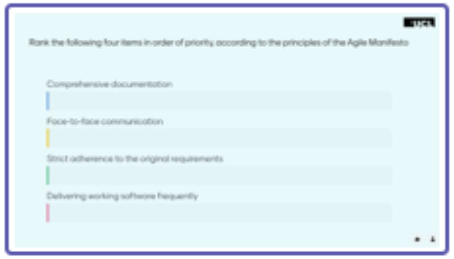
Delivering working software frequently

MB

Menti

ENGF0034 2025 - ...



Join at menti.com | use code 4411 5756

# Please ask your questions here

## Waiting for questions

Once questions are accepted by the moderator, they will show up here so that you can answer them one by one.

MB

**Menti**

ENGF0034 2025 - ...